

732A94 Advanced R programming

Lecture 6, part 3: Rcpp and memoization

Johan Alenlöv (Slides based on Leif Jonssons, Måns Magnusson and Krzysztof Bartoszek)

Linköpings Universitet

- Rcpp
 - Using C++ code in R
 - Mapping R data types to C++
 - Binary search example
- Memoization
 - `memoise`
 - Memoised Fibonacci

Questions?

Any questions from part 2?

- Needs a C++ compiler (see <http://adv-r.had.co.nz/Rcpp.html>)
- Often called **interfacing**
- The same can be done with Java and Fortran
- Extremely fast!

But just handle bottlenecks! Profile first (part 1). Rewriting code that takes 2% of the runtime buys you at most 2%.

$$F(n) = \begin{cases} n & \text{if } n < 2 \\ F(n-1) + F(n-2) & \text{otherwise} \end{cases}$$

The naive recursion recomputes the same values over and over – it is $O(\varphi^n)$. That makes it a nice, if artificial, benchmark.

```
fr <- function(n) {  
  if (n < 2) return(n)  
  fr(n - 1) + fr(n - 2)  
}  
system.time(fr(35))
```

```
##      user  system elapsed  
##    4.316    0.021    4.339
```

```
library(Rcpp)
cppFunction(code = '
int fcpp(int n) {
    if (n < 2) return(n);
    return(fcpp(n-1) + fcpp(n-2));
}
')
system.time(fcpp(35))
```

```
##      user  system elapsed
##    0.035    0.000    0.035
```

Same algorithm, same complexity – but a much smaller constant.

Rcpp chunks in R Markdown

knitr has an **Rcpp** engine, so you can write C++ directly in a chunk:

```
```{Rcpp}
#include <Rcpp.h>
using namespace Rcpp;

// [[Rcpp::export]]
double sum_cpp(NumericVector x) {
 double total = 0;
 for (int i = 0; i < x.size(); i++) total += x[i];
 return total;
}
```
```

The `// [[Rcpp::export]]` attribute is what makes the function visible from R. In a package you put the file in `src/` and run `Rcpp::compileAttributes()`.

Mapping R data types to C++

R and C++ support different data types. **Rcpp** implements the **RObject** class for the following mappings:

| R data type | Corresponding RObject class |
|------------------|------------------------------------|
| integer scalar | int |
| logical scalar | bool |
| double scalar | double |
| integer vector | IntegerVector |
| numeric vector | NumericVector |
| logical vector | LogicalVector |
| character vector | CharacterVector |
| numeric matrix | NumericMatrix |
| list | List |

Chapters 3–4 of *Seamless R and C++ Integration with Rcpp*, Eddebuettel (2013).

Beware: indexing starts at 0

```
cppFunction('double first_elem(NumericVector x) {  
  return x[0];    // NOT x[1]  
}')
```

```
first_elem(c(10, 20, 30))
```

```
## [1] 10
```

This is the single most common bug when moving R code to C++. The same goes for loop bounds: `for (int i = 0; i < n; i++)`.

- This example uses the **NumericVector** class to pass an R numeric vector as input to a C++ program
- Idea: search for an element in a sorted vector using binary search
- The R and C++ implementations are available in **binary_search_rcpp.R**

The example was not designed to show excellent performance gains through **Rcpp**, but rather to show how to pass a vector to an **Rcpp** program.

A wealth of performant examples: <https://gallery.rcpp.org/>

Binary search: the R version

```
binary_search_R <- function(x, target, n) {  
  L <- 1; R <- n  
  while (L <= R) {  
    m <- floor((L + R) / 2)  
    if (x[m] < target)      L <- m + 1  
    else if (x[m] > target) R <- m - 1  
    else                    return(m)  
  }  
  -1  
}  
x <- sort(rnorm(1000))  
binary_search_R(x, x[137], 1000)
```

```
## [1] 137
```

Binary search: the C++ version

```
library(Rcpp)
cppFunction('int binary_search_Cpp(NumericVector x,
                                   double target, int n) {

    int L = 0, R = n - 1;
    while (L <= R) {
        int m = floor((L + R) / 2);
        if (x[m] < target)      L = m + 1;
        else if (x[m] > target) R = m - 1;
        else                    return(m + 1);
    }
    return(-1);
}')

```

Note `int L = 0` and `R = n - 1`: C++ indexes from 0, so the returned position gets a + 1 on the way back to R.

Binary search: benchmarking

```
microbenchmark::microbenchmark(  
  R    = binary_search_R(x, x[137], 1000),  
  Cpp  = binary_search_Cpp(x, x[137], 1000),  
  times = 1000)
```

```
## Unit: nanoseconds
```

```
##  expr  min   lq     mean median   uq     max neval cld  
##    R 1517 1599 1633.481   1599 1640   6273  1000   a  
##   Cpp  656  738 1315.854    738  779 532016  1000   a
```

Memoization: a simple optimization technique

- An example of a general technique in optimization: **trading memory for computation**
- Memoization stores (caches) the results of function calls
- If called again with the same arguments, it returns the old value
- Depends on functional programming – the function must be **pure**
- Useful in recursive programming, instead of a manual look-up structure

```
library(memoise)
a <- function() runif(1)
replicate(3, a())
```

```
## [1] 0.61785224 0.07361558 0.24336226
```

```
b <- memoise(a)
replicate(3, b())
```

```
## [1] 0.02279261 0.02279261 0.02279261
```

Memoising an *impure* function changes its behaviour.

```
slow <- memoise(function() { Sys.sleep(1); runif(1) })  
system.time(slow())["elapsed"]    # computed
```

```
## elapsed  
##    1.006
```

```
system.time(slow())["elapsed"]    # from the cache
```

```
## elapsed  
##    0.001
```

```
invisible(forget(slow))           # empty the cache  
system.time(slow())["elapsed"]
```

```
## elapsed  
##    1.006
```

Memoised Fibonacci

```
frm <- memoise(function(n) {  
  if (n < 2) return(n)  
  frm(n - 1) + frm(n - 2)  
})  
system.time(frm(27))["elapsed"]
```

```
## elapsed  
##    0.004
```

The cache turns the exponential recursion into a linear one – the *algorithm* changed, not just the constant.

Memoisation of the C++ code did not improve the timing. Memoisation is sometimes an alternative to C++.

<https://www.inwt-statistics.com/read-blog/optimize-your-r-code-using-memoization.html>

When to reach for what

| Problem | Fix |
|-----------------------------------|----------------------------|
| Repeated identical calls | memoise |
| Loop over independent items | vectorise, parallel |
| Bad algorithm | a better algorithm |
| Tight scalar loop, no vector form | Rcpp |
| Growing objects in a loop | preallocate |

In that order. **Rcpp** is the last resort, not the first.

Questions?

See you next time!